

Cours 11 — Parcours et recherche dans un tableau

Semaine 11 ■ Manuel Hachette ch. 5 (§ 5.1, 5.2) ■ 2×1 h

Objectifs

- Écrire les algorithmes de référence : recherche, extremum, comptage, moyenne
- Compter le nombre d'opérations : premier contact avec le **coût linéaire**
- Justifier qu'un algorithme est correct (invariant, en français)

1. Les algorithmes de référence sur un tableau

Ces quatre motifs sont **exigibles au bac** : savoir les écrire sans modèle, les adapter, les reconnaître.

1.1 Recherche d'une valeur

```
def contient(t, x):  
    """Renvoie True si x apparaît dans le tableau t."""  
    for v in t:  
        if v == x:  
            return True  
    return False
```

Le `return False` est **après** la boucle : on ne peut conclure à l'absence qu'après avoir tout regardé. Variante indispensable : `indice(t, x)` qui renvoie la position de la première occurrence (ou `None`).

1.2 Extremum

```
def maximum(t):  
    """Renvoie le plus grand élément de t (t non vide)."""  
    maxi = t[0]  
    for v in t:  
        if v > maxi:  
            maxi = v  
    return maxi
```

Initialiser `maxi` avec `t[0]`, jamais avec 0 : le tableau `[-5, -2]` a pour maximum `-2`, pas 0. Et le tableau vide n'a pas de maximum — d'où la précondition « `t` non vide ».

1.3 Comptage et accumulation

```
def nb_occurrences(t, x):  
    """Renvoie le nombre d'apparitions de x dans t."""  
    compteur = 0  
    for v in t:  
        if v == x:  
            compteur = compteur + 1  
    return compteur
```

Même squelette pour la somme, la moyenne (somme puis division), le comptage sous condition.

2. Combien d'opérations ? Le coût linéaire

Pour un tableau de taille n , chacun de ces algorithmes examine chaque case **une fois** : environ n comparaisons. On dit que le coût est **linéaire**, noté $O(n)$: tableau 10 fois plus grand $\rightarrow$ temps 10 fois plus long.

La recherche peut s'arrêter dès que la valeur est trouvée : dans le **meilleur cas** 1 comparaison, dans le **pire cas** n . En algorithmique, on raisonne sur le **pire cas**.

Peut-on chercher *plus vite* que n ? Oui, si le tableau est **trié** — c'est la dichotomie, en semaine 15.

3. Pourquoi c'est correct ? L'invariant

Pour **maximum** : à *chaque tour de boucle*, **maxi** est le plus grand des éléments déjà examinés. Cette propriété est vraie au départ (**t**[0] seul examiné), reste vraie à chaque tour (on remplace si on trouve mieux), donc à la fin **maxi** est le plus grand de tous. C'est un **invariant de boucle** — on formalisera avec les tris (semaines 16-17).

À retenir

- Recherche : conclure « absent » seulement **après** la boucle.
- Extremum : initialiser avec **t**[0] et exiger **t** non vide.
- Compteur/accumulateur : initialiser avant, mettre à jour dans la boucle, renvoyer après.
- Un parcours simple coûte $O(n)$; on raisonne sur le pire cas.